



UNIVERSIDAD DE SANTIAGO DE CHILE

FACULTAD DE INGENIERÍA

Departamento de Ingeniería Informática

Informe laboratorio 3: Paralelismo con OpenMP

Profesor: Fernando Rannou Fuentes

Integrante: Xavier Muñoz Díaz

Santiago – Chile

Año 2023

Tabla de contenidos

1. Introducción.....	3
2. Desarrollo.....	4
3. Resultados.....	5
4. Conclusiones.....	11

1. Introducción

Las antenas del observatorio ALMA, ubicadas en el Desierto de Atacama, miden señales de radiofrecuencia en longitudes de onda milimétricas y submilimétricas, sin embargo, la señal capturada corresponde a la radiación de esa luz que produce al interactuar con el agua y el polvo que se encuentra alrededor de los objetos por lo que es necesario sintetizar la imagen, esto consiste en transformar esta señal de RF a una señal de intensidad.

Dado que el muestreo es irregular, el objetivo de este laboratorio es diseñar e implementar una aplicación paralela con OpenMP mediante un proceso llamado gridding, que se utiliza en síntesis de imágenes interferométricas, por lo que se debe transformar el plano (u, v) no grideado a uno que sí lo sea.

Para ello, se comienza a describir la solución con matrices globales y otra con matrices locales mediante el paralelismo y el uso de tareas. Luego se muestran los resultados obtenidos junto con el rendimiento computacional y para finalizar una breve conclusión del laboratorio.

2. Desarrollo

El proceso de gridding consiste en acumular todos los valores de los puntos (u, v) que caen en una celda (en su centro) por lo que se implementan dos soluciones, una con matrices locales y otra con matrices globales.

El programa recibe como parámetros el nombre del archivo de entrada y salida, delta x de la imagen en arco segundos, tamaño de la imagen, el número de chunks de lectura y el número de tareas.

A continuación, se describe brevemente las funciones más importantes del programa:

- void matrizCompartida: Abre el archivo de entrada, crea n tareas y mientras no se haya leído todo el archivo, cada tarea lee un chunk de líneas del archivo de entrada, luego se calcula la posición de la matriz que corresponde a la visibilidad y se acumula los resultados en las matrices globales. Para ello, recibe como entrada el nombre del archivo, el número de tareas que deben ser creadas, los chunks que debe leer del archivo cada tarea, el tamaño de la imagen y los deltas u y v.
- void matrizLocal: Realiza lo mismo que la función anterior, con la diferencia que cada tarea tiene su propia matriz para acumular el resultado obtenido. Una vez que se haya leído el archivo por completo y las tareas hayan terminado de acumular los resultados, se crea una sección crítica para que cada tarea vaya iterando en sus matrices locales y acumulando los resultados en matrices globales.
- main: Se encarga de tomar los parámetros ingresados por consola como el nombre del archivo de entrada y salida, el valor de delta x en arcosegundos, el tamaño de la imagen, los chunks de lectura y el número de tareas. Además, llama a diversas funciones para asignar memoria e inicializar en 0,0 las matrices globales, para realizar el gridding con los métodos explicados anteriormente, para normalizar las matrices, para escribir el resultado en los archivos de salida y para liberar la memoria de las matrices globales.

3. Resultados

Una vez compilado el código, se debe ejecutar la siguiente línea de comando:

```
./gridding -i hltau_completo_uv.csv -o datosgrideados -d 0,003 -N 2048 -c 1000 -t 1
```

Ambos algoritmos utilizan los mismos parámetros, en este caso se utilizan los siguientes valores:

- Nombre del archivo de entrada: hltau_completo_uv.csv
- Nombre del archivo de salida: datosgrideados
- Delta x (en arcosegundos): 0,003
- Tamaño de la imagen (matriz): 2048
- Número de chunks: 1000
- Número de tareas: toma los valores 1, 5, 10, 15, 30, 50, 100, 200 y 500

Además, para el cálculo del rendimiento computacional se utiliza la siguiente fórmula:

$Speedup = \frac{T_s}{T_p}$, donde T_s es el tiempo secuencial y T_p es el tiempo paralelo.

Para poder determinar el speedup, se realiza un promedio en las 10 ejecuciones del programa como se muestra en la Tabla 1.

Tareas	1		5		10	
	Paralelo		Paralelo		Paralelo	
	Con matrices compartida [s]	Con matrices locales [s]	Con matrices compartida [s]	Con matrices locales [s]	Con matrices compartida [s]	Con matrices locales [s]
	10,374773	10,345378	8,040094	6,036068	7,524491	6,226565
	9,939928	10,07592	7,727158	6,236573	7,699606	6,115792
	10,294242	10,600967	8,189043	6,525463	7,816504	6,241143
	10,567447	10,161174	8,094251	6,203547	7,709225	6,085185
	10,29645	10,429364	8,195026	6,278723	7,746775	6,064663
	10,565859	10,407211	7,773622	6,195048	7,76657	6,398473
	10,04658	10,088595	7,658217	6,161964	8,116808	6,160003
	10,376482	10,315816	7,841926	5,970573	7,683314	6,267358
	10,132962	10,185285	7,934207	5,769107	7,732649	6,096356
	10,405247	10,074155	7,755359	6,252045	7,678476	6,582278
Promedio	10,299997	10,268387	7,92089	6,162911	7,747442	6,223782

Tareas	15		30		50	
	Paralelo		Paralelo		Paralelo	
	Con matrices compartida [s]	Con matrices locales [s]	Con matrices compartida [s]	Con matrices locales [s]	Con matrices compartida [s]	Con matrices locales [s]
	7,905359	6,252446	8,341001	6,677798	8,512309	8,512309
	8,180207	6,258413	8,187762	6,727517	8,304061	7,011429
	8,243828	6,386523	8,20278	6,67786	8,770252	6,907852
	8,377318	6,476845	8,32076	6,48138	8,432613	6,971993
	8,676369	6,535772	8,273411	6,856625	8,445907	6,907854
	8,270346	6,168369	8,333354	6,594594	8,342917	7,281438
	8,617724	6,485733	8,405681	6,879495	8,382349	6,936646
	8,360828	6,233243	8,373866	6,604217	8,257926	6,990921
	8,562652	6,295319	8,552526	6,668131	8,21451	6,824475
	8,430279	6,308455	8,502233	6,525859	8,484849	7,236442
Promedio	8,362491	6,340112	8,349337	6,669348	8,414769	7,158136

Tareas	100		200		500	
	Paralelo		Paralelo		Paralelo	
	Con matrices compartida [s]	Con matrices locales [s]	Con matrices compartida [s]	Con matrices locales [s]	Con matrices compartida [s]	Con matrices locales [s]
	8,096766	8,352187	8,332029	8,365051	8,454583	19,178683
	8,223925	8,566654	8,365051	8,276133	8,31593	20,089323
	8,285959	8,087513	8,276133	8,353937	8,490869	19,087282
	8,458616	8,390976	8,353937	8,248392	8,327112	19,406806
	8,558001	8,272382	8,248392	8,306252	8,257685	19,04901
	8,252637	8,185308	8,306252	8,487481	8,284861	18,62811
	8,495907	8,586321	8,487481	8,887335	8,38967	19,51136
	8,339557	8,265198	8,887335	8,207518	8,190294	18,798443
	8,378692	8,312718	8,207518	8,347037	8,30534	17,420079

	8,492549	8,167349	8,347037	10,87497	8,226367	18,533251
Promedio	8,358261	8,318661	8,381117	8,635411	8,324271	18,970235

Tabla 1: Muestra los tiempos de ejecución paralela y secuencial (1 tarea), su promedio y su speedup con distinto número de tareas.

Fuente: Elaboración propia (2023).

De la tabla anterior, al graficar los tiempos promedios de ejecución para los dos algoritmos, se obtiene la Figura 1.

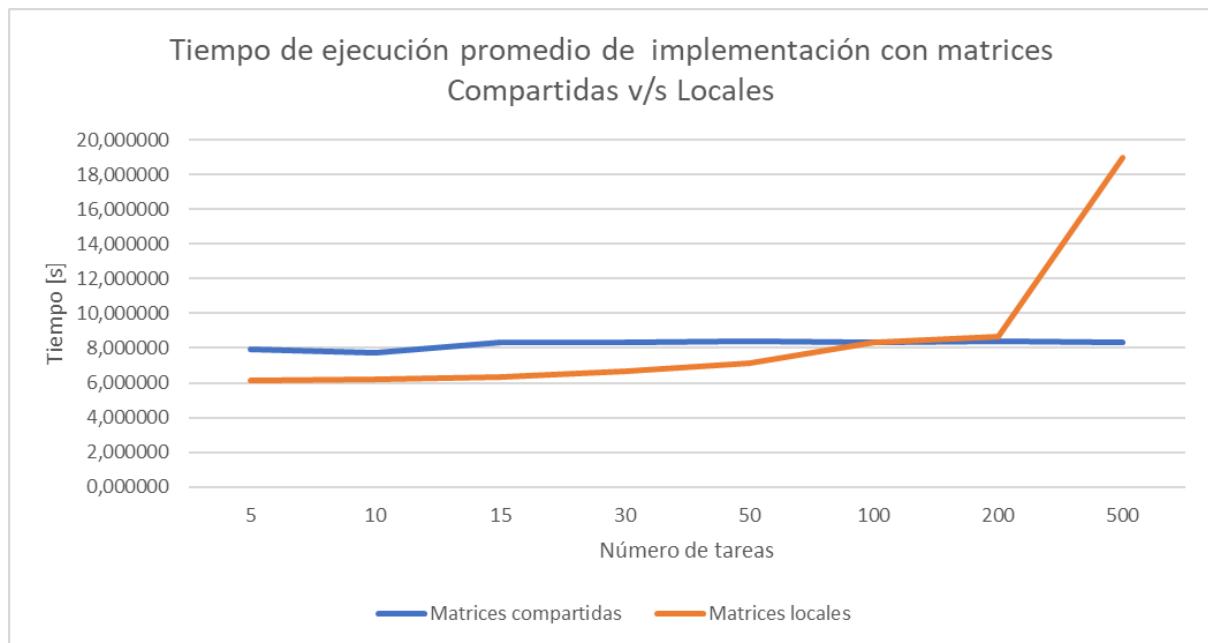


Figura 1: Muestra el tiempo promedio que se demora en ejecutar el algoritmo dependiendo del número de tareas y si es con matrices compartidas o locales.

Fuente: Elaboración propia (2023).

Además, al utilizar la fórmula del SpeedUp considerando el tiempo promedio de distintos números de tareas, se obtiene la Tabla 2, cuyos resultados se pueden ver gráficamente en la Figura 2.

Tareas	5	10	15	30	50	100	200	500
SpeedUp Matriz compartida	1,3004	1,3295	1,2317	1,2336	1,2240	1,2323	1,2290	1,2373
SpeedUp Matriz Local	1,6662	1,6499	1,6196	1,5396	1,4345	1,2344	1,1891	0,5413

Tabla 2: Cálculo de SpeedUp con el tiempo promedio que demora en ejecutar los dos algoritmos con distintos números de tareas.

Fuente: Elaboración propia (2023).

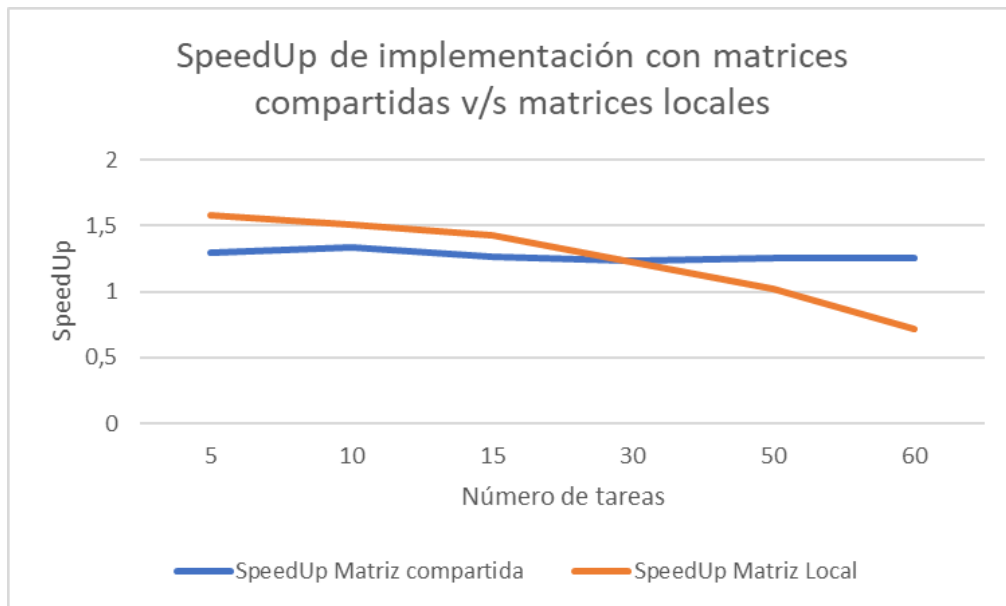


Figura 2: Muestra el SpeedUp dependiendo del número de tareas para el algoritmo con matrices locales y matrices compartidas.

Fuente: Elaboración propia (2023).

Finalmente, en las Figuras 3 y 4 se puede apreciar el resultado de la ejecución de los algoritmos mencionados anteriormente. Estas figuras se obtienen como resultado al ejecutar el código de MATLAB que se observa en la Figura 5.

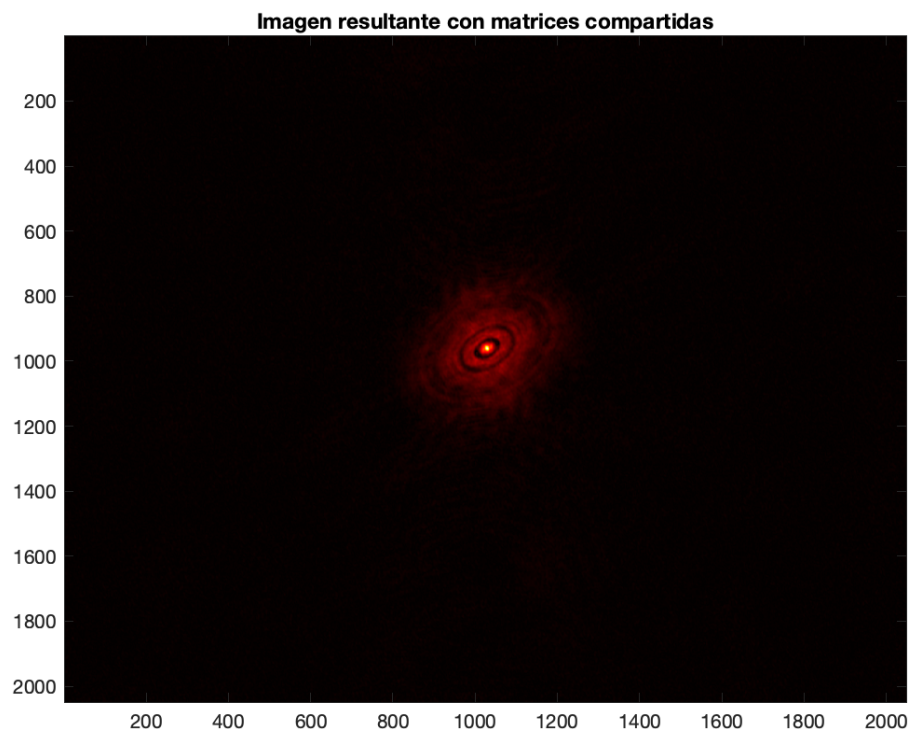


Figura 1: Resultado del grideo de los datos con matrices compartidas con la ayuda de MATLAB.

Fuente: Elaboración propia (2023).

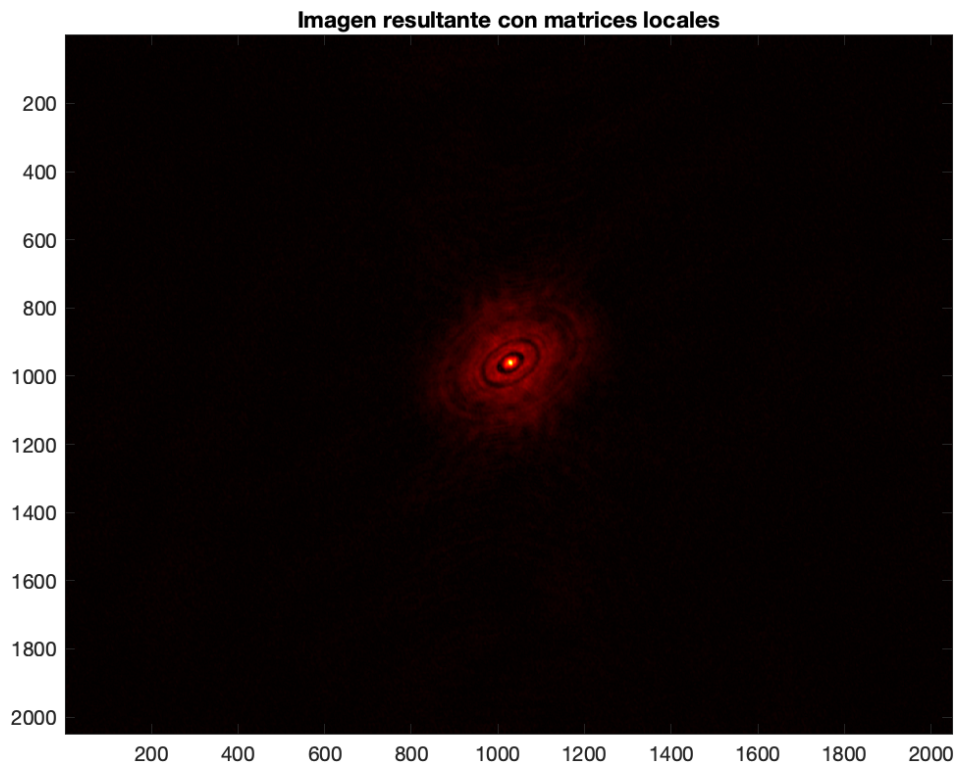


Figura 2: Resultado del grideo de los datos con matrices locales con la ayuda de MATLAB.

Fuente: Elaboración propia (2023).

```

% Abre los archivos de datos grideados con matrices compartidas
f1 = fopen("datosgrideadosr.raw", "rb");
f2 = fopen("datosgrideadosi.raw", "rb");

% Abre los archivos de datos grideados con matrices locales
f3 = fopen("datosgrideadosr_local.raw", "rb");
f4 = fopen("datosgrideadosi_local.raw", "rb");

% Lee los archivos con matrices globales que poseen valores tipo float de doble precisión
s1 = fread(f1, "double");
s2 = fread(f2, "double");

% Lee los archivos con matrices locales que poseen valores tipo float de doble precisión
s3 = fread(f3, "double");
s4 = fread(f4, "double");
|
% Cierra los archivos
fclose(f1);
fclose(f2);
fclose(f3);
fclose(f4);

% Toma los vectores creados anteriormente y los transforma en una matriz
re1 = reshape(s1, 2048, 2048);
im1 = reshape(s2, 2048, 2048);
re2 = reshape(s3, 2048, 2048);
im2 = reshape(s4, 2048, 2048);

% Crea vectores complejos
v1 = complex(re1, im1);
v2 = complex(re2, im2);

% Realiza la transformada inversa de Fourier
I = fftshift(ifft2(v1));
J = fftshift(ifft2(v2));

% Muestra la primera imagen en una figura
figure;
imagesc(abs(I));
colormap('hot');
title('Imagen resultante con matrices compartidas');

% Muestra la segunda imagen en otra figura
figure;
imagesc(abs(J));
colormap('hot');
title('Imagen resultante con matrices locales');

```

*Figura 3: Código en MATLAB para ver la imagen resultante.
Fuente: Elaboración propia (2023).*

4. Conclusiones

Considerando que las imágenes resultantes corresponden a la solución esperada las cuales son obtenidas utilizando algoritmos con programación paralela mediante el uso de OpenMP, se cumple el objetivo del laboratorio.

Por otro lado, según la Figura 1, el tiempo que se demora en ejecutar el algoritmo que posee matrices compartidas, tiende a subir levemente y luego a bajar cuando el número de tareas incrementa pero el cambio es muy pequeño por lo que se puede decir que el valor es casi constante. Por otro lado, el algoritmo que posee matrices locales, cuando el número de tareas es menor a 100 tiene un tiempo promedio de ejecución mucho menor que el otro algoritmo aunque a medida que aumenta el número de tareas, el tiempo de ejecución también lo hace, sin embargo, cuando el número de tareas está entre 100 y 200 aumenta muy poco y cuando ya son 500 el tiempo de ejecución aumenta a más del doble que cuando este tenía 200 tareas.

Además, al observar la Tabla 2, se observa que el algoritmo con matrices compartidas tiende a aumentar y disminuir dependiendo del número de tareas que tenga pero se mantiene casi constante, por lo que se puede concluir que sobre 30 tareas el tiempo de ejecución no varía mucho y probablemente sea conveniente trabajar con 10 tareas considerando que esta obtuvo el mejor SpeedUp.

En cambio el algoritmo que utiliza matrices locales, presenta una mayor mejora en comparación al otro algoritmo cuando el número de tareas es mayor a 1 pero es menor cuando tiene aproximadamente 30 tareas, es más, a medida que se aumenta el número de tareas su SpeedUp disminuye llegando incluso a ser una peor solución que el algoritmo secuencial, lo cual se observa gráficamente en la Figura 2. En consecuencia es conveniente usar entre 5 a 10 tareas dado que estas obtuvieron el mejor SpeedUp.